

Build Pipeline: .c/.h (AUTOSAR RTE/SWC) → TASKING ctc (compiler) → .o → TASKING ltc (linker) → .elf → ielftool → .hex / .S19

1. What is an ELF File? (In AURIX Context)

ELF (Executable and Linkable Format) is the standard binary container produced by your TASKING TriCore toolchain. In the Embedded SW build it is the master debug artifact — the .hex and .S19 files that get flashed onto the Microcontrollers, are derived from this ELF by the compilers like TASKING ielftool post-processor. The ELF itself is never flashed directly but contains everything the debugger (Lauterbach TRACE32, iSYSTEM winIDEA, AURIX Development Studio) needs to perform source-level debugging.

Format class	ELF32 — TriCore is a 32-bit Harvard-ish architecture
Byte order	Little-endian (TriCore / Infineon standard)
Machine type	e_machine = 0x006D (EM_TRICORE)
OS/ABI	ELFOSABI_NONE (0x00) — bare-metal, no OS ABI
ELF type	ET_EXEC (executable) for the final linked image
Entry point	Reset vector address — typically _START or __cstart in TASKING crt0
AUTOSAR relevance	Contains all BSW, MCAL, RTE, SWC, OS code + symbol + DWARF debug info
Related outputs	.hex (Intel HEX for flash), .S19 (Motorola SREC), .map (linker map)

Visu-IT! ELF Viewer 1.3	
C:\Tasks\1.Debugging\TC_Aurix_Blinky.elf	
Class	ELFCLASS32
Data	LSB
HeaderVersion	1
Type	Executable file
Entry Address	0xA0000000
Machine	Siemens TriCore embedded processor
Flags	524288
Version	Current version
Header Size	52
Number of ProgramHeaders	46
Start of ProgramHaders	52
Size of ProgramHeaders	32
Number of SectionHeaders	1633
Start of SectionHeaders	783580
SectionHeaderEntrySize	40
SectionHeaderStringTableIndex	1632

2. Who Generates the ELF? (TASKING Toolchain)

The TASKING compiler TriCore toolchain generates the ELF in two stages, plus an optional post-processing stage:

Tool	Executable	Role in ELF creation
TASKING C/C++ Compiler	ctc.exe / cctc	Compiles AUTOSAR SWC, BSW, MCAL .c/.cpp → relocatable ELF .o objects with DWARF debug sections
TASKING Assembler	astc.exe	Assembles .src/.asm startup code (cstart.src, trap tables) → ELF .o objects
TASKING Linker	ltc.exe	Links all .o + libraries using .lsl linker script → final ELF executable
TASKING ielftool	ielftool.exe	Post-processes ELF → generates .hex / .S19 / .bin; does NOT change the .elf
AUTOSAR Build (Tresos/EB)	make / cmake / Bazel	Orchestrates the above; feeds -D macros, include paths, and .lsl scripts per variant

TASKING note: The linker script (.lsl) maps AUTOSAR memory regions (PFlash0-5, DSRAM0-5, PSRAM, DFLASH) onto the TC4D7XP / TC397XP physical addresses.

These mappings are reflected in the ELF program headers (LOAD segments).

3. When is the ELF Generated?

Build Stage	Trigger	ELF artifact produced
Compilation	Each .c/.cpp change	.o — relocatable ELF per translation unit; contains .text, .data, .debug_*
Linking	All objects ready / ltc runs	Final .elf — all segments resolved to physical addresses from .lsl
Post-link	ielftool runs on .elf	.hex / .S19 derived; .elf itself is not modified at this point
CI / Nightly build	Automated pipeline	Versioned .elf archived alongside .map for regression comparison

4. What Information Does the ELF Contain? (AURIX-Specific)

The final linked ELF from the TASKING ltc for TC4D7XP / TC397XP contains the following major sections and segments:

Section / Segment	Type	AURIX / AUTOSAR Specific Content
.text / .ztext	Code (PFlash)	All AUTOSAR BSW, MCAL, RTE, SWC compiled functions. .ztext is 'near' code for short-call range on TriCore
.data / .zdata	Init data (DSRAM)	Initialised global/static variables. Copied from PFlash LMA to DSRAM VMA by __cstart
.bss / .zbss	Zero-init (DSRAM)	Zero-initialised globals (AUTOSAR NvM shadow buffers, OS stacks, RTE buffers)

ELF File Concept

Reference: AUTOSAR, Infineon AURIX TriCore MCUs, TASKING Compiler

6-Core TriCore
TC1.6.2/TC1.8
Architecture

Bhabani Sankar
Mishra

.rodata / .zrodata	Constants (PFlash)	AUTOSAR DEM/DTC tables, Com config structs, calibration defaults, string literals
.CPU0_init..CPU5_init	Per-core code (PFlash)	TASKING generates per-core sections; each of 6 TriCore cores has its own init & private RAM sections
.psram_text	PSRAM code	Time-critical ISRs / AUTOSAR OS hooks placed in Program SRAM for zero-wait execution
DFLASH / EEPROM sections	Data Flash	NvM/EEPROM emulation blocks; LMA in DFLASH address space
.csa (CSA region)	Context Save Area	TriCore-specific: hardware CSA linked list area per core; defined in .lsl
.istack / .ustack	Stacks	Interrupt and user stacks per core; size set by TASKING linker symbols __ISTACK_SIZE
.symtab / .strtab	Symbol table	All function & variable symbols with addresses; used by debugger and ielftool
.debug_info (DWARF4)	Debug info	AUTOSAR SWC/RTE variable types, function params, inline functions (RTE_READ/RTE_WRITE macros)
.debug_line	Line tables	Source line ↔ PFlash address mapping for every .c file in the build
.debug_frame (CFI)	Unwind tables	Call-frame info for stack unwinding (critical for multi-core crash analysis)
PT_LOAD segments	Program headers	One LOAD segment per memory region (PFlash0..5, DSRAM0..5, PSRAM) — used by flash programmer
.note.tasking.build	Build note	TASKING embeds build ID, compiler version, options — useful for traceability

5. What to Debug Using the ELF? (AURIX / AUTOSAR Scenarios)

Debug Scenario	ELF Information Used	Tool & How
Core trap / BISR crash	PC at trap, .debug_line, .debug_frame, CSA chain	TRACE32 d.load.elf + register dump; tricore.cmm script unwinds CSA
Stack overflow (per core)	.istack/.ustack symbols, linker stack size symbols	Check SP vs __ISTACK / __USTACK_CPU0..5 boundaries via nm or TRACE32 v.info
AUTOSAR OS task runout	OS task stack sections, OSEK/OS config symbols	Verify OS stack sizes in .bss per core; compare with OsStackSize config
Memory map overflow (PFlash full)	.text + .rodata size vs PFlash0..5 capacity	readelf -S / .map file; compare segment LMA ranges to TC397XP 16MB PFlash
Linker symbol mismatch	__START_OF_XXX / __END_OF_XXX symbols wrong VMA	nm --defined-only grep __START; cross-check with .lsl region definitions
Multi-core data sharing bug	Shared DSRAM section VMA, cache/MPU attributes in ELF notes	Verify shared section in one core's local DSRAM vs DLMU global SRAM

ELF File Concept

Reference: AUTOSAR, Infineon AURIX TriCore MCUs, TASKING Compiler

6-Core TriCore
TC1.6.2/TC1.8
Architecture

Bhabani Sankar
Mishra

RTE variable not updating	AUTOSAR RTE buffer symbol, .data LMA vs VMA	Check LMA/VMA split: is __cstart correctly copying .data from PFlash to RAM?
Wrong function address (calibration)	.rodata CalParam symbol address	addr2line or nm; verify symbol lives in PFlash not accidentally in RAM
Build traceability / version mismatch	.note.tasking.build section, build-id	readelf --notes; compare compiler version embedded in ELF with expected
Binary size regression (CI)	Section sizes: .text, .rodata, .data, .bss	size tool or bloaty; alert if .text grows > threshold between builds

6. How to Read the ELF -

The TASKING toolchain ships its own ELF utilities (prefixed tricore-), but GNU binutils cross-compiled for TriCore also work. In some TASKING compiler versions, if “objdump” does not support, the commands using **“elfdump.exe and hldumputc.exe”** can be used when we want to extract the segments, sections, debug info from the elf file, as shown below.

```
$ hldumputc -a myfile.elf > debug_extract.txt  
  
$ elfdump -p myfile.elf > elf_extract.txt  
$ elfdump -c myfile.elf >> elf_extract.txt
```

Commands below assume tricore- prefix; replace with arm-none-eabi- or readelf if using generic GNU tools, Or can be **referred to the corresponding compiler toolchain documentations**.

Goal	Command / Tool	Output / Notes
ELF header (arch, entry pt)	tricore-readelf -h app.elf	Shows e_machine=0x6D (TriCore), entry address (reset vector)
All sections + sizes	tricore-readelf -S app.elf	Lists .text, .rodata, .CPU0_init, .csa, .symtab, .debug_* with VMA/LMA/size
Load segments (flash regions)	tricore-readelf -l app.elf	Shows PT_LOAD entries mapping PFlash0..5, DSRAM, PSRAM — use to verify .lsl
Symbol table (all functions)	tricore-nm --demangle app.elf	Function names with PFlash addresses; grep for your SWC function
Symbol sizes (biggest code)	tricore-nm --size-sort -r app.elf head -30	Top 30 largest functions — useful for code size reduction
Disassemble .text	tricore-objdump -d app.elf less	TriCore assembly with symbol labels; use -S for interleaved C source (needs DWARF)
Disassemble single function	tricore-objdump -d --disassemble=Rte_Read_P1_S1 app.elf	Disassemble just one AUTOSAR RTE function
Address → source line	tricore-addr2line -e app.elf -f 0x80001234	Returns filename:line for a PFlash trap address — essential for crash diagnosis
Section sizes summary	tricore-size app.elf	Quick .text/.data/.bss totals; run after every build to detect size regressions
DWARF debug info	tricore-readelf --debug-dump=info app.elf less	Dumps all type/variable/function DWARF DIEs — shows AUTOSAR struct layouts

ELF File Concept |

Reference: AUTOSAR, Infineon AURIX TriCore MCUs, TASKING Compiler

6-Core TriCore
TC1.6.2/TC1.8
Architecture

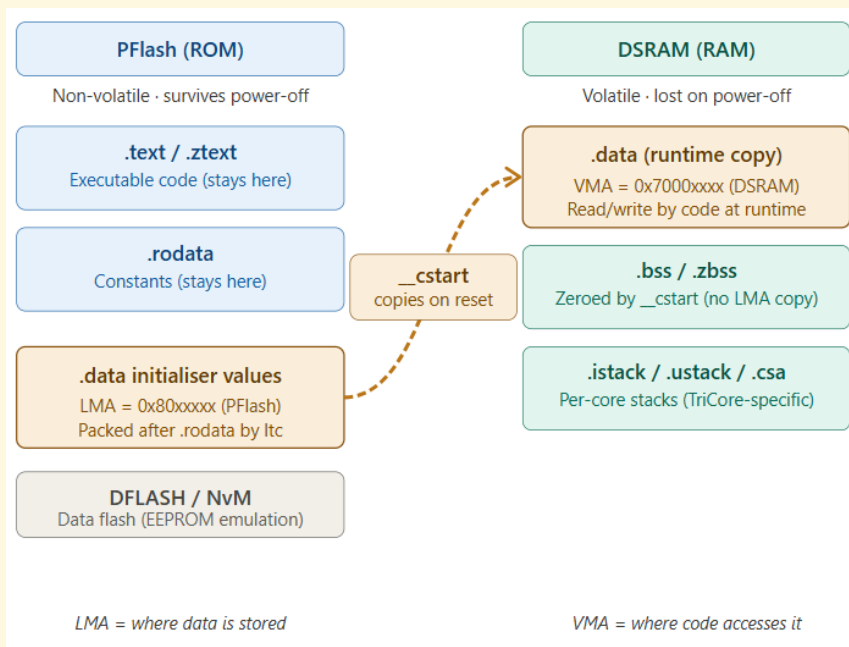
Bhabani Sankar
Mishra

DWARF line table	<code>tricore-readelf --debug-dump=line app.elf</code>	Shows address-to-source line mapping for every compilation unit
Build notes (compiler ver)	<code>tricore-readelf --notes app.elf</code>	Reads <code>.note.tasking.build</code> — confirms exact TASKING version used
Extract raw section bytes	<code>tricore-objcopy -O binary --only-section=.text app.elf text.bin</code>	Useful for CRC verification of PFlash content
TRACE32 (Lauterbach) — full load	<code>d.load.elf app.elf /nocode</code>	Loads symbols only (flash already programmed); enables src-level debug on all 6 cores
AURIX Dev Studio / iSYSTEM	File > Load Symbols > app.elf	GUI debugger; auto-reads DWARF4 for variable watch, AUTOSAR OS task view

TASKING-specific tip: Use `tricore-readelf -S app.elf | grep -E 'CPU[0-5]'` to list all per-core sections and verify each of the 6 TriCore cores (CPU0–CPU5 on TC4D7XP) has correct private DSRAM assignments as defined in your `.lsl` linker script.

LMA ≠ VMA warning: On AURIX, initialised `.data` sections have LMA in PFlash and VMA in DSRAM.

`tricore-readelf -S` will show both.



Key concept:

LMA (Load Memory Address) = physical location in PFlash where the ELF stores the initial values.

VMA (Virtual Memory Address) = address your C code uses at runtime — in DSRAM on AURIX.

At reset, `__cstart` reads from LMA and writes to VMA, bridging the two.

Debugger workflow: 1) Flash `.hex` via Infineon Memtool or TRACE32 → 2) Load `.elf` symbols: `d.load.elf app.elf /nocode` → 3) Set breakpoints by function name or source line → 4) Use `v.info address <symbol>` to inspect AUTOSAR ports/runnable variables.